## In Conclusion

I hope that you have now grasped the underlying reasons for the SOLID principles. They exist to make change (which is inevitable) less impactful and easier to manage. By decoupling components from each other and shifting dependencies from classes to interfaces (or abstract classes), we reduce the impact that change requests have throughout the system. If we continuously evaluate classes in terms of the human actors that make use of them, we can anticipate change before it happens and ensure that any particular change request impacts the smallest amount of code possible.

You should now be aware that large interfaces are to be viewed with (extreme) suspicion, and likewise any instances of subclass members being overloaded with error handling code (or even no code at all) is a clear indicator of underlying design problems. By considering the open-closed principle and Liskov substitution, new classes that you design should be more robust and cause fewer maintenance headaches down the line. It is good practice to always consider the engineers that will work on your code in the future and try to make their lives as easy as possible – remember that you might be one of them!

# Design Patterns

The SOLID principles are an excellent first step to designing better object-oriented software; classes that are designed with those principles in mind will generally be easier to understand, integrate with each other, and maintain throughout the life of the code. Building on these ideas of integration and reuse, we come to *design patterns*. It is rare to encounter genuinely novel problems in software engineering; most of the time the code that you write professionally will be treading well-worn paths: authenticating requests, sending emails, transforming raw data into something browsers can use… and so on. The generally agreed landmark work in the software design patterns space is *Design Patterns: Elements of Reusable Object-Oriented Software*, written by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (the so-called "Gang of Four" – often abbreviated to *GoF*), first published in 1994. Another notable text is *Patterns of Enterprise Application Architecture*, by Martin Fowler and first published in 2022.

Design patterns are reusable templates for class design, intended to solve common problem cases. Using patterns can speed up the development process: rather than needing to create novel solutions, one can simply choose an appropriate pattern, which